📅 Week 1-2: HTML & CSS (Day-by-Day Learning Plan)

📌 **Goal**: Build a strong foundation in **HTML & CSS** and create responsive designs.
🎯 **Outcome**: By the end of 2 weeks, you will be able to structure a webpage and style it effectively using modern CSS techniques.

🔰 Week 1: HTML Fundamentals & Basic CSS

📅 Day 1: Introduction to HTML & Basic Structure

📚 **Topics to Cover**:

- What is HTML?
- Structure of an HTML Document (`<!DOCTYPE html>`, `<html>`, `<head>`, `<body>`)
- Common HTML Tags (`<h1>-<h6>`, `<p>`, `<a>`, `<br>`, `<hr>`)
- Lists (`<ul>`, `<ol>`, `<li>`)

📝 **Practice Tasks**: ✔️ Create a basic **HTML page** with a heading, paragraph, and links.

📅 Day 2: HTML Semantic Elements & Forms

📚 **Topics to Cover**:

- Semantic HTML (`<header>`, `<nav>`, `<section>`, `<article>`, `<footer>`)
- Forms (`<form>`, `<input>`, `<label>`, `<button>`, `<textarea>`, `<select>`)

📝 **Practice Tasks**: ✔️ Create a **simple contact form** with name, email, message, and submit button.

📅 Day 3: Tables & Multimedia in HTML

📚 **Topics to Cover**:

- Tables (`<table>`, `<tr>`, `<td>`, `<th>`, `<thead>`, `<tbody>`)
- Embedding Media (`<img>`, `<audio>`, `<video>`, `<iframe>`)

📝 **Practice Tasks**: ✔️ Create a **pricing table** for a website with at least three columns (Basic, Pro, Premium).
✔️ Embed a YouTube video and an audio file on a webpage.

📅 Day 4: Introduction to CSS & Selectors

📚 **Topics to Cover**:

- Ways to apply CSS (Inline, Internal, External)
- CSS Selectors (`element`, `.class`, `#id`, `*`, `> +`, `~`)
- Colors, Backgrounds, and Borders

📝 **Practice Tasks**: ✔️ Style your **previous HTML forms and tables** using CSS.
✔️ Try different **CSS selectors** on a sample webpage.

📅 Day 5: CSS Box Model & Typography

📚 **Topics to Cover**:

- Box Model (Margin, Padding, Border, Content)
- Width & Height Properties
- Fonts, Font Sizes, Google Fonts
- Text Styling (`text-align`, `line-height`, `letter-spacing`)

📝 **Practice Tasks**: ✔️ Create a **styled blog page** with different font styles and proper spacing.

🔰 Week 2: CSS Layouts & Responsive Design

📅 Day 6: CSS Flexbox

📚 **Topics to Cover**:

- Flexbox Container (`display: flex`, `justify-content`, `align-items`)
- Flexbox Items (`flex-grow`, `flex-shrink`, `order`)
- Nested Flexbox

📝 **Practice Tasks**: ✔️ Build a **responsive navbar** using Flexbox.

📅 Day 7: CSS Grid

📚 **Topics to Cover**:

- Grid Container (`display: grid`, `grid-template-columns`, `grid-template-rows`)
- Grid Items (`grid-column`, `grid-row`)
- Grid vs Flexbox

📝 **Practice Tasks**: ✔️ Design a **3-column product listing page** using CSS Grid.

📅 Day 8: Positioning & Z-Index

📚 **Topics to Cover**:

- CSS Positioning (`static`, `relative`, `absolute`, `fixed`, `sticky`)
- Z-Index & Overlapping Elements

📝 **Practice Tasks**: ✔️ Create a **sticky header and a floating button** on a webpage.

📅 Day 9: Transitions & Animations

📚 **Topics to Cover**:

- CSS Transitions (`transition-property`, `transition-duration`, `transition-timing-function`)
- CSS Animations (`@keyframes`, `animation-name`, `animation-duration`)

📝 **Practice Tasks**: ✔️ Add a **hover effect** with transition to buttons.
✔️ Create a **simple loading animation** using CSS animations.

📅 Day 10: Responsive Design with Media Queries

📚 **Topics to Cover**:

- Responsive Design Principles
- Media Queries (`@media (max-width: 768px)`)
- Mobile-First vs Desktop-First Approach

📝 **Practice Tasks**: ✔️ Make your **previous projects mobile-friendly** using media queries.

📅 Day 11-12: Mini Projects

🎯 **Goal**: Apply everything learned so far to build real projects.

✅ **Project 1: Portfolio Website**

- Create a personal **portfolio website** with a homepage, about, projects, and contact section.
- Use **CSS Flexbox/Grid for layout** and **media queries for responsiveness**.

✅ **Project 2: Responsive Landing Page**

- Build a **landing page** for a business or product with a navbar, hero section, and call-to-action button.

🎯 Final Outcome After 2 Weeks:

✅ You will have **a strong foundation in HTML & CSS**.
✅ You will be able to create **responsive layouts** with Flexbox & Grid.
✅ You will have **2 completed projects** for your portfolio.

🚀 **Next Step: Move on to JavaScript!** 🎉

📅 Week 3-4: JavaScript Fundamentals & DOM Manipulation

📌 **Goal**: Learn JavaScript from basics to DOM manipulation and build interactive web pages.
🎯 **Outcome**: By the end of 2 weeks, you will be able to write JavaScript code, manipulate the DOM, and handle user interactions.

🔰 Week 3: JavaScript Basics

📅 Day 1: Introduction to JavaScript & Variables

📚 **Topics to Cover**:

- What is JavaScript?
- How JavaScript runs in the browser
- Variables (`var`, `let`, `const`)
- Data Types (String, Number, Boolean, Undefined, Null, Object, Array)

📝 **Practice Tasks**:
✔️ Create a program that stores your name, age, and city in variables and logs them to the console.

📅 Day 2: Operators & Conditionals

📚 **Topics to Cover**:

- Arithmetic Operators (`+`, `-`, `*`, `/`, `%`, `**`)
- Comparison Operators (`==`, `===`, `!=`, `<`, `>`, `<=`, `>=`)
- Logical Operators (`&&`, `||`, `!`)
- Conditional Statements (`if`, `else`, `switch`)

📝 **Practice Tasks**:
✔️ Build a **grading system** where a user enters marks, and the program outputs the grade (A, B, C, D, F).

📅 Day 3: Loops & Functions

📚 **Topics to Cover**:

- Loops (`for`, `while`, `do-while`)
- Functions (`function` keyword, arrow functions, parameters, return values)

📝 **Practice Tasks**:
✔️ Write a function to **calculate the factorial** of a given number using a loop.

📅 Day 4: Arrays & Objects

📚 **Topics to Cover**:

- Arrays (`push()`, `pop()`, `shift()`, `unshift()`, `splice()`, `slice()`)
- Looping through Arrays (`for`, `forEach`, `map`)
- Objects (`key-value pairs`, `dot notation`, `bracket notation`)

📝 **Practice Tasks**:
✔️ Create an **array of student objects**, each with a name and score. Write a function to find the highest scorer.

📅 Day 5: ES6 Features (Modern JavaScript)

📚 **Topics to Cover**:

- `let` vs `const` vs `var`
- Template Literals
- Destructuring (`const {name, age} = person`)
- Spread & Rest Operators
- Arrow Functions
- Default Parameters

📝 **Practice Tasks**:
✔️ Create an object and use **destructuring** to extract values.
✔️ Use **spread operator** to merge two arrays.

🔰 Week 4: DOM Manipulation & Events

📅 Day 6: Introduction to the DOM

📚 **Topics to Cover**:

- What is the DOM?
- Selecting Elements (`getElementById`, `querySelector`, `querySelectorAll`)
- Changing Content (`textContent`, `innerHTML`)

📝 **Practice Tasks**:
✔️ Write JavaScript code to change the text of a `<p>` tag when a button is clicked.

📅 Day 7: Handling User Events

📚 **Topics to Cover**:

- Events (`click`, `mouseover`, `keydown`, `keyup`)
- `addEventListener()` method
- Event Object (`event.target`)

📝 **Practice Tasks**:
✔️ Build a **color changer** where clicking a button changes the background color of the page.

📅 Day 8: DOM Manipulation - Adding & Removing Elements

📚 **Topics to Cover**:

- Creating Elements (`document.createElement`)
- Appending Elements (`appendChild`, `insertBefore`)
- Removing Elements (`removeChild`)

📝 **Practice Tasks**:
✔️ Build a **to-do list** where users can add and remove tasks dynamically.

📅 Day 9: Working with Forms & Local Storage

📚 **Topics to Cover**:

- Getting Input Values (`document.getElementById().value`)
- Preventing Form Submission (`event.preventDefault()`)
- Local Storage (`localStorage.setItem`, `localStorage.getItem`)

📝 **Practice Tasks**:
✔️ Create a **user registration form** that saves user data in Local Storage.

📅 Day 10: Async JavaScript - Callbacks, Promises, Async/Await

📚 **Topics to Cover**:

- Callbacks & Callback Hell
- Promises (`.then()`, `.catch()`)
- Async/Await

📝 **Practice Tasks**:
✔️ Create a **fake API call simulation** using Promises and `setTimeout()`.

📅 Day 11-12: Mini Projects

🎯 **Goal**: Apply everything learned so far to build real projects.

✅ **Project 1: Interactive To-Do List**

- Users can **add, mark complete, and delete tasks**.
- Data is **stored in Local Storage**.

✅ **Project 2: Weather App** (Using OpenWeather API)

- Users enter a city, and the app fetches & displays weather info.
- Uses **Fetch API & Async/Await**.

🎯 Final Outcome After 2 Weeks:

✅ You will understand **JavaScript fundamentals & DOM manipulation**.
✅ You will be able to **handle user interactions & work with APIs**.
✅ You will have **2 completed projects** for your portfolio.

🚀 **Next Step: Move on to Advanced JavaScript & React!** 🎉

📅 Week 5-6: Advanced JavaScript & React Basics

📌 **Goal**: Learn advanced JavaScript concepts like OOP, ES6+ features, and start React with components, state, and props.
🎯 **Outcome**: By the end of these two weeks, you'll be able to build a **basic React project** with functional components and state management.

🔰 Week 5: Advanced JavaScript & API Integration

📅 Day 1: JavaScript Object-Oriented Programming (OOP)

📚 **Topics to Cover**:

- Objects & Prototypes
- Constructor Functions
- ES6 Classes (`class`, `constructor`, `extends`, `super`)

📝 **Practice Tasks**:
✔️ Create a **Person class** with properties like `name`, `age`, and a method `introduce()`.
✔️ Create a `Student` class that extends `Person` and adds a `grade` property.

📅 Day 2: JavaScript Modules & Local Storage

📚 **Topics to Cover**:

- `export` & `import` for modules
- Local Storage (`setItem`, `getItem`, `removeItem`)
- Session Storage

📝 **Practice Tasks**:
✔️ Create a **note-taking app** where users can add, edit, and delete notes, storing them in Local Storage.

📅 Day 3: Fetch API & Handling API Data

📚 **Topics to Cover**:

- Fetching data from APIs (`fetch()`, `.then()`, `.catch()`)
- Handling JSON data (`JSON.stringify()`, `JSON.parse()`)
- Error handling (`try...catch`)

📝 **Practice Tasks**:
✔️ Build a **user list app** that fetches user data from `https://jsonplaceholder.typicode.com/users` and displays it.

📅 Day 4: Async/Await & Axios

📚 **Topics to Cover**:

- Async/Await (`async function myFunction()`)
- Using `Axios` for API calls
- Error Handling (`try-catch`)

📝 **Practice Tasks**:
✔️ Convert the **previous API project** to use Async/Await and Axios.

📅 Day 5: Mini Project - API-Based Web App

🎯 **Goal**: Apply Fetch API/Async-Await in a project.

✅ **Project: Random User Generator**

- Fetch random users from `https://randomuser.me/api/`.
- Display name, age, and profile picture.

🔰 Week 6: Introduction to React

📅 Day 6: Setting Up React & JSX

📚 **Topics to Cover**:

- What is React? Why use it?
- Installing React (`create-react-app` or Vite)

- JSX Basics
- Rendering Elements

📝 **Practice Tasks**:
✔️ Set up a **new React project** and render an `<h1>Hello, React!</h1>`.

📅 Day 7: Components & Props

📚 **Topics to Cover**:

- Functional Components
- Props & Passing Data
- Children Prop

📝 **Practice Tasks**:
✔️ Create a **Profile Card** component that takes `name`, `age`, and `image` as props.

📅 Day 8: State & useState Hook

📚 **Topics to Cover**:

- State in React (`useState`)
- Updating State
- Handling Events

📝 **Practice Tasks**:
✔️ Build a **counter app** with buttons to increase/decrease/reset the count.

📅 Day 9: Handling Forms in React

📚 **Topics to Cover**:

- Handling Form Inputs (`onChange`, `useState`)
- Controlled vs Uncontrolled Components
- Form Submission (`onSubmit`)

📝 **Practice Tasks**:
✔️ Create a **simple login form** that takes `username` and `password`, and logs them to the console on submit.

📅 Day 10: Fetching Data in React (useEffect & Axios)

📚 **Topics to Cover**:

- Fetch API inside React (`useEffect`)
- Displaying API Data in Components
- Axios with React

📝 **Practice Tasks**:
✔️ Build a **User List app** in React that fetches users from `https://jsonplaceholder.typicode.com/users`.

📅 Day 11-12: Mini Project - React App

🎯 **Goal**: Build a small React project.

✅ **Project: Weather App (React + API)**

- Users enter a city name, and the app fetches weather details using an API.
- Display temperature, weather condition, and an icon.

🎯 Final Outcome After 2 Weeks:

✅ You will understand **advanced JavaScript & React basics**.
✅ You will know how to **handle APIs in JavaScript & React**.
✅ You will have **2 React mini-projects** for your portfolio.

🚀 **Next Step: Move on to React Advanced Concepts & Node.js!** 🎉

📅 Week 7-8: Advanced React & Introduction to Node.js

📌 **Goal**: Learn advanced React concepts like routing, context API, and state management. Then, start with backend development using Node.js and Express.
🎯 **Outcome**: By the end of these two weeks, you'll be able to **build a full-stack application with React and Node.js**.

🔰 Week 7: Advanced React Concepts

📅 Day 1: React Router (Navigation in React Apps)

📚 **Topics to Cover**:

- Why do we need React Router?
- Installing React Router (`react-router-dom`)
- Creating Routes (`<BrowserRouter>`, `<Routes>`, `<Route path="/" element={<Home />} />`)
- Navigating Between Pages (`Link`, `useNavigate()`)

📝 **Practice Tasks**:
✔️ Create a **multi-page website** with a home page, about page, and contact page using React Router.

📅 Day 2: React Context API (Global State Management)

📚 **Topics to Cover**:

- Why use Context API?
- Creating a Context
- Using `useContext()` Hook
- Passing Data Globally

📝 **Practice Tasks**:
✔️ Build a **theme toggler** using Context API that switches between light and dark mode.

📅 Day 3: React Hooks - useReducer & useRef

📚 **Topics to Cover**:

- `useReducer` for state management
- `useRef` for referencing DOM elements
- When to use `useReducer` over `useState`

📝 **Practice Tasks**:
✔️ Convert a **counter app** from `useState` to `useReducer`.
✔️ Create a **focus input field** using `useRef`.

📅 Day 4: React Forms & Validation (Formik & Yup)

📚 **Topics to Cover**:

- Controlled vs Uncontrolled Forms
- Using **Formik** for form handling
- Using **Yup** for form validation

📝 **Practice Tasks**:
✔️ Build a **sign-up form** with validation using Formik & Yup.

📅 Day 5: Mini Project - Advanced React App

🎯 **Goal**: Apply React Router, Context API, and Hooks.

✅ **Project: Blog App**

- Users can **view blog posts**.
- Admin panel to **add & delete blogs**.
- Uses **React Router, Context API, and Local Storage**.

🔰 Week 8: Introduction to Node.js & Express.js

📅 Day 6: Introduction to Node.js

📚 **Topics to Cover**:

- What is Node.js?
- Setting up Node.js (`node -v`, `npm init -y`)
- Running JavaScript outside the browser (`console.log("Hello, Node.js")`)
- Creating & Running a Simple Node.js File (`node app.js`)

📝 **Practice Tasks**:
✔️ Create a **Node.js script** that logs your name, age, and current date.

📅 Day 7: Introduction to Express.js (Backend Framework)

📚 **Topics to Cover**:

- What is Express.js?
- Setting up an Express Server (`npm install express`)
- Creating Routes (`app.get()`, `app.post()`)

📝 **Practice Tasks**:
✔️ Build a **simple Express.js server** with routes for home, about, and contact pages.

📅 Day 8: Middleware & Request Handling

📚 **Topics to Cover**:

- What is Middleware?
- Built-in Middleware (`express.json()`, `express.urlencoded()`)
- Custom Middleware

📝 **Practice Tasks**:
✔️ Create an **Express server** that logs every request using custom middleware.

📅 Day 9: REST API & CRUD Operations

📚 **Topics to Cover**:

- What is REST API?
- Creating a simple CRUD API (`GET`, `POST`, `PUT`, `DELETE`)
- Handling JSON data

📝 **Practice Tasks**:
✔️ Build a **Books API** with routes to **add, update, delete, and fetch books**.

📅 Day 10: Connecting Node.js to MongoDB (Mongoose)

📚 **Topics to Cover**:

- Installing MongoDB & Mongoose (`npm install mongoose`)
- Connecting Node.js to MongoDB
- Defining a Schema & Model
- Performing CRUD operations on MongoDB

📝 **Practice Tasks**:
✔️ Create a **MongoDB database** and store book details in it.

📅 Day 11-12: Mini Project - Full-Stack App

🎯 **Goal**: Build a full-stack MERN app with React & Node.js.

✅ **Project: Task Manager App**

- Frontend: React (Create, Update, Delete tasks)
- Backend: Node.js + Express (API to handle tasks)
- Database: MongoDB (Store tasks)

🎯 Final Outcome After 2 Weeks:

✅ You will understand **advanced React concepts** (Routing, Context API, useReducer).
✅ You will be able to **build a REST API using Node.js & Express**.
✅ You will have **a full-stack project (React + Node.js + MongoDB)** in your portfolio.

🚀 **Next Step: Move on to Authentication & Deployment!** 🎉

📅 Week 9-10: Authentication, Security & Deployment

📌 **Goal**: Implement authentication (JWT & OAuth), improve security, and deploy a full-stack MERN application.
🎯 **Outcome**: By the end of these two weeks, you'll have **user authentication in your MERN app and a fully deployed project**.

🔰 Week 9: Authentication & Security in MERN

📅 Day 1: Introduction to Authentication (Session vs Token-based)

📚 **Topics to Cover**:

- What is authentication?
- Difference between Session-based and Token-based Authentication
- How JWT (JSON Web Token) works

📝 **Practice Tasks**:
✔️ Draw a flowchart explaining **JWT authentication process**.

📅 Day 2: User Authentication with JWT

📚 **Topics to Cover**:

- Installing JWT (`npm install jsonwebtoken bcryptjs dotenv`)
- Creating user model (MongoDB + Mongoose)
- Hashing passwords with `bcryptjs`
- Generating and verifying JWT tokens

📝 **Practice Tasks**:
✔️ Build a **login and signup API** using Node.js, Express, and MongoDB.

📅 Day 3: Protecting Routes & Middleware

📚 **Topics to Cover**:

- Protecting routes using JWT (`req.headers.authorization`)
- Creating authentication middleware
- Handling token expiration & errors

📝 **Practice Tasks**:
✔️ Protect a **dashboard route** that only authenticated users can access.

📅 Day 4: User Authentication in React (Frontend)

📚 **Topics to Cover**:

- Sending login/signup requests from React
- Storing JWT in Local Storage
- Using Axios for authentication requests
- Protecting routes in React (Private Routes)

📝 **Practice Tasks**:
✔️ Build a **React authentication system** with login, signup, and protected pages.

📅 Day 5: Role-Based Authentication (Admin & User)

📚 **Topics to Cover**:

- Assigning roles (admin/user)
- Restricting access based on roles
- Implementing role-based middleware

📝 **Practice Tasks**:
✔️ Create an **admin panel** that only admin users can access.

🔰 Week 10: Security, Optimization & Deployment

📅 Day 6: Securing Your Application

📚 **Topics to Cover**:

- Preventing SQL Injection & NoSQL Injection
- Using Helmet.js (`npm install helmet`)
- Preventing Cross-Site Scripting (XSS) & Cross-Site Request Forgery (CSRF)

📝 **Practice Tasks**:
✔️ Secure your **MERN app** using `helmet` and `cors`.

📅 Day 7: Error Handling & Logging

📚 **Topics to Cover**:

- Global error handling in Express (`app.use((err, req, res, next) => {})`)
- Logging errors (`winston`, `morgan`)
- Handling client-side errors in React

📝 **Practice Tasks**:
✔️ Implement error logging in your backend using `morgan`.

📅 Day 8: Payment Integration (Optional - Stripe/PayPal)

📚 **Topics to Cover**:

- Introduction to online payments
- Setting up Stripe (`npm install stripe`)
- Creating checkout sessions

📝 **Practice Tasks**:
✔️ Build a **simple checkout page** using Stripe API.

📅 Day 9: Preparing for Deployment

📚 **Topics to Cover**:

- Optimizing React for production (`npm run build`)
- Using `dotenv` for environment variables
- Connecting to a cloud database (MongoDB Atlas)

📝 **Practice Tasks**:
✔️ Move your **MongoDB database** to MongoDB Atlas.

📅 Day 10: Deploying Backend to Render/Heroku

📚 **Topics to Cover**:

- Deploying Express backend on **Render or Heroku**
- Setting up environmental variables
- Testing API endpoints on the live server

📝 **Practice Tasks**:
✔️ Deploy your **Node.js API** and test it online.

📅 Day 11: Deploying Frontend to Vercel/Netlify

📚 **Topics to Cover**:

- Deploying React on **Vercel/Netlify**
- Handling environment variables in React
- Connecting frontend with backend

📝 **Practice Tasks**:
✔️ Deploy your **full-stack MERN app** and test it live.

📅 Day 12: Final Full-Stack Project (Complete Authentication System)

🎯 **Goal**: Deploy a complete **MERN authentication system** with user login, protected routes, and database integration.

✅ **Final Project Features:**

- User authentication with **JWT**
- Role-based access (User/Admin)
- Secure **backend with Express & MongoDB**
- **Frontend React UI** for authentication
- **Deployed on Vercel & Render**

🎯 Final Outcome After 2 Weeks:

✅ You will know **how to implement authentication using JWT**.
✅ You will understand **backend security practices**.
✅ You will have a **fully deployed MERN authentication project**.

🚀 **Next Step: Move on to Advanced Backend Development & Scaling!** 🎉

📅 Week 11-12: Advanced Backend Development & Scaling MERN Apps

📌 **Goal**: Learn advanced backend concepts, implement WebSockets, caching, and optimize your MERN app for scalability.
🎯 **Outcome**: By the end of these two weeks, you'll be able to build a **real-time, high-performance MERN application** with optimized database queries and WebSockets.

🔰 Week 11: Advanced Backend Concepts & Optimization

📅 Day 1: File Uploads with Multer & Cloud Storage

📚 **Topics to Cover**:

- Using Multer for file uploads (`npm install multer`)
- Uploading files to Cloudinary/AWS S3
- Handling file validation & security

📝 **Practice Tasks**:
✔️ Build a **profile image upload API** that stores images in Cloudinary.

📅 Day 2: WebSockets & Real-Time Communication (Socket.io)

📚 **Topics to Cover**:

- Introduction to WebSockets
- Setting up Socket.io in Express (`npm install socket.io`)
- Creating a real-time chat feature

📝 **Practice Tasks**:
✔️ Build a **basic real-time chat app** using Socket.io.

📅 Day 3: Background Jobs with Node.js (Bull Queue & Worker Threads)

📚 **Topics to Cover**:

- Using `Bull` (`npm install bull`) for background jobs
- Running tasks in the background (e.g., sending emails)
- Worker threads for heavy computations

📝 **Practice Tasks**:
✔️ Implement an **email notification system** using Bull Queue.

📅 Day 4: Caching with Redis for Performance Optimization

📚 **Topics to Cover**:

- What is caching, and why use Redis?
- Installing Redis (`npm install redis`)
- Implementing caching for API responses

📝 **Practice Tasks**:
✔️ Cache API responses in Redis to **improve performance**.

📅 Day 5: Mini Project - Real-Time Task Manager

🎯 **Goal**: Apply WebSockets, Redis, and Multer in a full-stack project.

✅ **Project: Real-Time Task Manager App**

- Users can **create tasks**.
- **Tasks update in real-time** for all users.
- File uploads & Redis caching for fast access.

🔰 Week 12: Scaling & Advanced Deployment

📅 Day 6: Load Balancing & Horizontal Scaling

📚 **Topics to Cover**:

- What is Load Balancing?
- Using Nginx as a reverse proxy
- Scaling Node.js apps using PM2 (`npm install pm2 -g`)

📝 **Practice Tasks**:
✔️ Deploy a **load-balanced Node.js app** with PM2 & Nginx.

📅 Day 7: Microservices Architecture

📚 **Topics to Cover**:

- What are microservices?
- Breaking a monolithic app into microservices
- Communication between microservices (REST & Message Queues)

📝 **Practice Tasks**:
✔️ Build a **simple microservice-based architecture** for user authentication.

📅 Day 8: Docker & Containerization

📚 **Topics to Cover**:

- What is Docker?
- Creating a Docker container for a MERN app
- Running MongoDB in a Docker container

📝 **Practice Tasks**:
✔️ **Dockerize a MERN app** and run it inside a container.

📅 Day 9: CI/CD (Continuous Integration & Deployment)

📚 **Topics to Cover**:

- Introduction to CI/CD
- Setting up **GitHub Actions** for auto-deployment
- Deploying using **Vercel, Render, or AWS**

📝 **Practice Tasks**:
✔️ Implement **GitHub Actions for auto-deployment**.

📅 Day 10: Database Optimization & Indexing

📚 **Topics to Cover**:

- Using **Indexes** in MongoDB for faster queries
- Optimizing queries with `explain()`
- MongoDB Aggregation Framework

📝 **Practice Tasks**:
✔️ Optimize **MERN app database queries** using indexes.

📅 Day 11-12: Final Full-Stack Project - Scalable MERN App

🎯 **Goal**: Deploy a **high-performance, scalable MERN app** with real-time features.

✅ **Final Project Features:**

- **User authentication & role-based access**
- **Real-time notifications (Socket.io + Redis)**
- **Cloud storage for file uploads**
- **Dockerized & optimized for production**

🎯 Final Outcome After 2 Weeks:

✅ You will understand **real-time communication using WebSockets**.
✅ You will know **how to scale & optimize a backend**.
✅ You will have a **fully optimized & containerized MERN app**.

🚀 **Next Step: Master DevOps & System Design!** 🎉

📅 Week 13-14: DevOps, System Design & Advanced Scaling

📌 **Goal**: Learn how to deploy, monitor, and scale large-scale MERN applications. Master DevOps tools, system design principles, and cloud computing.
🎯 **Outcome**: By the end of these two weeks, you'll be able to **design, deploy, and manage a highly scalable production-ready application**.

🔰 Week 13: DevOps & Cloud Deployment

📅 Day 1: Introduction to Cloud Computing & Deployment Strategies

📚 **Topics to Cover**:

- Understanding **IaaS (AWS, GCP, Azure) vs PaaS (Vercel, Render, Heroku)**
- Choosing the right deployment strategy: **Single Server vs Multi-Server vs Kubernetes**
- Introduction to AWS services: **EC2, S3, RDS, Lambda**

📝 **Practice Tasks**:
✔️ **Compare different cloud providers** and decide which one to use for your MERN app.

📅 Day 2: Deploying MERN Stack on AWS EC2

📚 **Topics to Cover**:

- Setting up an **EC2 instance**
- Installing **Node.js, MongoDB, Nginx** on EC2
- Running a **PM2 process manager** to keep the server running
- Using **Nginx as a reverse proxy**

📝 **Practice Tasks**:
✔️ Deploy a **MERN app on AWS EC2** using PM2 and Nginx.

📅 Day 3: Docker & Kubernetes for Containerized Deployment

📚 **Topics to Cover**:

- Creating a **Dockerfile** for a MERN app
- Running a **MongoDB container** using Docker
- Introduction to **Kubernetes & Cluster Orchestration**

📝 **Practice Tasks**:
✔️ **Containerize a MERN app** using Docker and deploy it locally.

📅 Day 4: Load Balancing & Auto-Scaling

📚 **Topics to Cover**:

- What is a **Load Balancer**?
- Setting up **AWS Load Balancer / Nginx Load Balancer**
- Auto-scaling with **AWS Auto Scaling Groups**

📝 **Practice Tasks**:
✔️ Implement **a load-balanced MERN app** with multiple backend instances.

📅 Day 5: Mini Project - Scalable Cloud Deployment

🎯 **Goal**: Deploy a **highly scalable MERN app** using AWS, Docker, and Load Balancers.

✅ **Project: Scalable Task Manager**

- **Frontend & Backend hosted on AWS**
- **Dockerized containers running on EC2**
- **Load balancing & auto-scaling enabled**

🔰 Week 14: System Design & Performance Optimization

📅 Day 6: Introduction to System Design

📚 **Topics to Cover**:

- Understanding **Scalability, Availability, and Reliability**
- **Monolithic vs Microservices Architecture**
- API Gateway & Rate Limiting

📝 **Practice Tasks**:
✔️ Design a **scalable architecture** for a social media app.

📅 Day 7: Database Sharding & Replication

📚 **Topics to Cover**:

- **Vertical vs Horizontal Scaling**
- **MongoDB Sharding** for large databases
- **Database Replication & High Availability**

📝 **Practice Tasks**:
✔️ Implement **MongoDB replication** for high availability.

📅 Day 8: Caching Strategies & Redis Optimization

📚 **Topics to Cover**:

- **In-Memory Caching with Redis**
- **Cache Invalidation Strategies** (LRU, LFU)
- **CDN (Cloudflare) for Faster Content Delivery**

📝 **Practice Tasks**:
✔️ Implement **Redis caching** in an API for fast response times.

📅 Day 9: Message Queues & Event-Driven Architecture

📚 **Topics to Cover**:

- What are **Message Queues (RabbitMQ, Kafka)**?
- **Event-Driven Microservices**
- **Decoupling services using a Queue**

📝 **Practice Tasks**:
✔️ Implement a **message queue for email notifications**.

📅 Day 10: Security & API Rate Limiting

📚 **Topics to Cover**:

- **Rate Limiting using Express & Redis**
- **DDOS Protection & Web Application Firewalls (WAFs)**
- **Securing API Keys & Environment Variables**

📝 **Practice Tasks**:
✔️ Implement **rate limiting in your API** to prevent abuse.

📅 Day 11-12: Final Capstone Project - Enterprise Level MERN App

🎯 **Goal**: Build & deploy a **fully scalable, optimized, and secure MERN app**.

✅ **Final Project Features:**

- **Microservices-based architecture**
- **Dockerized & deployed on AWS**
- **Redis caching & MongoDB replication**
- **Load balancer for auto-scaling**
- **Real-time updates with WebSockets**

🎯 Final Outcome After 2 Weeks:

✅ You will understand **cloud deployment & auto-scaling**.
✅ You will be able to **design large-scale systems**.
✅ You will have a **highly scalable & optimized MERN app**.